CONTENT MODEL

Amendments & SQL Reference — v3

Pomegranate Marketing · KG CMS · Supabase

This document amends content-model-v3.docx. It replaces Section C (Database Migration SQL) and amends the Schema Architecture in Section B. All changes are backwards-compatible additions or column renames applied via safe ALTER TABLE IF NOT EXISTS / ADD IF NOT EXISTS statements.

1. Summary of Agreed Decisions

#	Decision	Detail
D1	**associates = brands/partners/software only**	The associates table is for external brands, partner organisations, and software platforms (e.g. WordPress, Google, SemRush). It is NOT for human authors or team members. Any previous references to associates as an author source are superseded.
D2	**New public.people table**	Create a dedicated people table for authors, reviewers, and team members. This table is the source of Person schema for BlogPosting and the team section on the About page.
D3	**blog_posts author + reviewer FKs**	Add blog_posts.author_person_id and blog_posts.reviewer_person_id as nullable FK columns pointing to people.id. Legacy author_name / author_url / author_image_url are retained as transitional fallback fields.
D4	**faq_list (jsonb) = single FAQ standard**	The column name faq_list (jsonb array of objects) is the single standard across pages, blog_posts, services, industries, free_tools. Replaces: pages.faqs, blog_posts."FAQs", pseo_page_instances.unique_faqs. Backfill then drop legacy columns.
D5	**schema_json_ld (jsonb) = single schema cache standard**	The column name schema_json_ld (jsonb) is the single standard across all content tables. Replaces generated_schema_markup ARRAY columns. pseo_page_instances already has this column — remove generated_schema_markup there once backfill confirmed.
D6	**industries columns**	industries must include canonical_url, keyword_cycling_blocks, faq_list, schema_json_ld. seo_title and seo_meta_desc already present.
D7	**free_tools columns**	free_tools must include seo_title, seo_meta_desc, canonical_url, keyword_cycling_blocks, faq_list, schema_json_ld.
D8	**businesses = default publisher**	businesses remains the default publisher for all blog content. people provides the Person author data. associates supports brand / sponsor / provider roles where appropriate.

2. Column Name Changes Quick Reference

Any references in the content model templates, schema architecture, or application code that use the old column names must be updated to the new standard names.

Table	Was	Now (amended)
pages	`pages.faqs (ARRAY)`	`pages.faq_list (jsonb DEFAULT '[]')`
pages	`pages.generated_schema_markup (ARRAY)`	`pages.schema_json_ld (jsonb)`
blog_posts	`blog_posts."FAQs" (ARRAY)`	`blog_posts.faq_list (jsonb DEFAULT '[]')`
blog_posts	`blog_posts.generated_schema_markup (ARRAY)`	`blog_posts.schema_json_ld (jsonb)`
pseo_page_instances	`pseo_page_instances.unique_faqs (jsonb) — keep as reference`	`pseo_page_instances.faq_list (jsonb DEFAULT '[]')`
pseo_page_instances	`pseo_page_instances.generated_schema_markup (ARRAY) — drop after backfill`	`pseo_page_instances.schema_json_ld (jsonb) — already present`

Table	Was	Now (amended)
services	`(no faq column existed)`	`services.faq_list (jsonb DEFAULT '[]')`
services	`(no schema cache existed)`	`services.schema_json_ld (jsonb)`
industries	`industries.faqs (if existed)`	`industries.faq_list (jsonb DEFAULT '[]')`
industries	`(no schema cache existed)`	`industries.schema_json_ld (jsonb)`
free_tools	`(no faq column existed)`	`free_tools.faq_list (jsonb DEFAULT '[]')`
free_tools	`(no schema cache existed)`	`free_tools.schema_json_ld (jsonb)`

3. New Table: public.people

This table is the canonical source for human author, reviewer, and team member data. It supports Person schema generation for BlogPosting and enables the team section on the About page.

3.1 Field Reference

Field Name	Field Type	Required?	Notes
id	uuid, PK	Yes	uuid_generate_v4()
business_id	uuid, FK → businesses	Yes	Which business this person belongs to
full_name	text	Yes	Displayed name and used in Person.name schema
slug	text	Yes	URL-safe identifier — unique per business_id
job_title	text	No	Used in Person.jobTitle schema
bio	text	No	Long-form bio for About/profile pages
short_bio	text	No	1–2 sentence bio for author card component
profile_image_url	text	No	Used in Person.image schema
profile_image_alt	text	No	Alt text for profile image
email	text	No	Contact email — consider whether to expose publicly
website_url	text	No	Personal site or portfolio — Person.url
linkedin_url	text	No	LinkedIn profile
sameas_urls	jsonb DEFAULT '[]'	No	Array of canonical profile URLs for Person.sameAs in schema
social_links	jsonb	No	Full platform metadata (handle, follower count etc.) — not used in schema directly
expertise_areas	text[] DEFAULT '{}'	No	Array of topic tags — used in expanded author/expert component
credentials	text[] DEFAULT '{}'	No	Qualifications, certifications, accreditations
works_for_business_id	uuid, FK → businesses	No	If person works for a different entity to the site business
is_author	boolean DEFAULT true	No	Can be selected as blog post author
is_reviewer	boolean DEFAULT false	No	Can be selected as blog post reviewer
is_team_member	boolean DEFAULT true	No	Shown in About page team section
published	boolean DEFAULT false	No	Controls visibility in frontend team displays
seo_title	text	No	For individual profile pages if they exist
seo_meta_desc	text	No	

Field Name	Field Type	Required?	Notes
canonical_url	text	No	
in_language	text DEFAULT 'en-GB'	No	Language of this person's content
schema_json_ld	jsonb	No	Generated Person schema cache
created_at	timestamptz DEFAULT now()	Yes	
last_updated	timestamptz DEFAULT now()	Yes	

3.2 CREATE TABLE SQL

```
-- ============================================================
-- NEW TABLE: public.people
-- Authors, reviewers, and team members
-- ============================================================
CREATE TABLE public.people (
  id                      uuid NOT NULL DEFAULT uuid_generate_v4(),
  business_id             uuid NOT NULL,
  full_name               text NOT NULL,
  slug                    text NOT NULL,
  job_title               text,
  bio                     text,
  short_bio               text,
  profile_image_url       text,
  profile_image_alt       text,
  email                   text,
  website_url             text,
  linkedin_url            text,
  sameas_urls             jsonb DEFAULT '[]'::jsonb,
  social_links            jsonb,
  expertise_areas         text[] DEFAULT '{}'::text[],
  credentials             text[] DEFAULT '{}'::text[],
  works_for_business_id   uuid,
  is_author               boolean DEFAULT true,
  is_reviewer             boolean DEFAULT false,
  is_team_member          boolean DEFAULT true,
  published               boolean DEFAULT false,
  seo_title               text,
  seo_meta_desc           text,
  canonical_url           text,
  in_language             text DEFAULT 'en-GB'::text,
  schema_json_ld          jsonb,
  created_at              timestamptz DEFAULT now(),
  last_updated            timestamptz DEFAULT now(),
```

```
  CONSTRAINT people_pkey PRIMARY KEY (id),
  CONSTRAINT people_business_id_fkey
    FOREIGN KEY (business_id) REFERENCES public.businesses(id),
  CONSTRAINT people_works_for_fkey
    FOREIGN KEY (works_for_business_id) REFERENCES public.businesses(id),
  CONSTRAINT people_slug_business_unique UNIQUE (business_id, slug)
);
```

3.3 blog_posts FK columns SQL

```
-- Add author and reviewer FK columns to blog_posts
-- Nullable — existing rows are unaffected
ALTER TABLE public.blog_posts
  ADD COLUMN IF NOT EXISTS author_person_id   uuid,
  ADD COLUMN IF NOT EXISTS reviewer_person_id uuid;

ALTER TABLE public.blog_posts
  ADD CONSTRAINT blog_posts_author_person_fkey
    FOREIGN KEY (author_person_id) REFERENCES public.people(id),
  ADD CONSTRAINT blog_posts_reviewer_person_fkey
    FOREIGN KEY (reviewer_person_id) REFERENCES public.people(id);

COMMENT ON COLUMN public.blog_posts.author_person_id IS
  'FK to people table. Primary source for Person/author schema. NULL allowed — fallback to
author_name / author_url / author_image_url.';
COMMENT ON COLUMN public.blog_posts.reviewer_person_id IS
  'FK to people table for reviewer/editor. Optional. Used in review schema if present.';
```

4. Standardisation SQL — faq_list & schema_json_ld

Run order:
Run this AFTER the people table creation and blog_posts FK columns. Use ADD COLUMN IF NOT EXISTS throughout — safe to re-run.

4.1 Add standard columns

```
-- ============================================================
-- STEP 1: Add faq_list jsonb to all content tables
-- ============================================================
ALTER TABLE public.pages
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

ALTER TABLE public.blog_posts
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

ALTER TABLE public.industries
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

ALTER TABLE public.free_tools
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

ALTER TABLE public.pseo_page_instances
  ADD COLUMN IF NOT EXISTS faq_list jsonb DEFAULT '[]'::jsonb;

-- ============================================================
-- STEP 2: Add schema_json_ld jsonb to all content tables
-- (pseo_page_instances already has this — IF NOT EXISTS skips it)
-- ============================================================
ALTER TABLE public.pages
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;

ALTER TABLE public.blog_posts
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;

ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;

ALTER TABLE public.industries
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;
```

```
ALTER TABLE public.free_tools
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;

ALTER TABLE public.pseo_page_instances
  ADD COLUMN IF NOT EXISTS schema_json_ld jsonb;
```

4.2 Backfill faq_list from legacy columns

Validate the legacy column data type and shape before running backfill. The exact expression depends on whether the legacy column stored plain text, JSON strings, or structured arrays.

```
-- ============================================================
-- STEP 3: Backfill faq_list from legacy FAQ columns
-- Adapt cast expressions to match your actual data shapes.
-- ============================================================

-- pages.faqs → pages.faq_list
-- If pages.faqs is already jsonb:
UPDATE public.pages
  SET faq_list = faqs
  WHERE faqs IS NOT NULL AND jsonb_typeof(faqs) = 'array' AND faq_list = '[]'::jsonb;

-- blog_posts."FAQs" → blog_posts.faq_list
-- Note the quoted uppercase column name
UPDATE public.blog_posts
  SET faq_list = "FAQs"
  WHERE "FAQs" IS NOT NULL AND faq_list = '[]'::jsonb;

-- pseo_page_instances.unique_faqs → faq_list
UPDATE public.pseo_page_instances
  SET faq_list = unique_faqs
  WHERE unique_faqs IS NOT NULL AND unique_faqs != '[]'::jsonb
    AND faq_list = '[]'::jsonb;

-- ============================================================
-- STEP 4: Validate backfill
-- Run these and confirm counts match your expectations
-- ============================================================
SELECT COUNT(*) FROM public.pages WHERE faq_list != '[]'::jsonb;
SELECT COUNT(*) FROM public.blog_posts WHERE faq_list != '[]'::jsonb;
SELECT COUNT(*) FROM public.pseo_page_instances WHERE faq_list != '[]'::jsonb;
```

4.3 Drop legacy columns (run after app compatibility confirmed)

CAUTION — staged drop:

Do NOT run these DROP statements until (a) the schema generators in Phase 3 have been updated to read from faq_list only, (b) all CMS panels write to faq_list only, and (c) backfill validation passes. These are commented out intentionally.

```
-- Run these only after confirming full application compatibility
-- and successful backfill validation (Step 4 above).

-- ALTER TABLE public.pages DROP COLUMN IF EXISTS faqs;
-- ALTER TABLE public.blog_posts DROP COLUMN IF EXISTS "FAQs";
-- ALTER TABLE public.pseo_page_instances DROP COLUMN IF EXISTS unique_faqs;

-- generated_schema_markup deprecation
-- Run only after Phase 3 schema generators are complete and
-- all schema_json_ld columns have been backfilled.

-- ALTER TABLE public.pages DROP COLUMN IF EXISTS generated_schema_markup;
-- ALTER TABLE public.blog_posts DROP COLUMN IF EXISTS generated_schema_markup;
-- ALTER TABLE public.pseo_page_instances DROP COLUMN IF EXISTS generated_schema_markup;
```

5. Amended Section C — Full Migration SQL (replaces content-model-v3 Section C)

This section replaces Section C of content-model-v3.docx entirely. Run in order. Safe to re-run — all statements use IF NOT EXISTS. Backup your database before starting.

```
-- ============================================================
-- MIGRATION: Content Model v3 (amended)
-- Pomegranate Marketing — KG CMS Supabase
-- Run in Supabase SQL editor. Backup first.
-- ============================================================

-- 1. businesses: website schema config + sameAs
ALTER TABLE public.businesses
  ADD COLUMN IF NOT EXISTS website_schema_config jsonb DEFAULT '{}'::jsonb,
  ADD COLUMN IF NOT EXISTS sameas_urls            text[] DEFAULT '{}'::text[];

-- 2. pages: cycling blocks + canonical + faq_list + schema_json_ld
ALTER TABLE public.pages
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS schema_json_ld         jsonb;

-- 3. blog_posts: cycling blocks + canonical + faq_list + schema + author FKs
ALTER TABLE public.blog_posts
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS schema_json_ld         jsonb,
  ADD COLUMN IF NOT EXISTS author_person_id        uuid,
  ADD COLUMN IF NOT EXISTS reviewer_person_id      uuid;

-- 4. pseo_page_instances: cycling blocks + canonical + faq_list
--    (schema_json_ld already exists on this table)
ALTER TABLE public.pseo_page_instances
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb;

-- 5. services: long description, SEO fields, cycling, faq_list, schema
ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS long_description       text,
  ADD COLUMN IF NOT EXISTS seo_title              text,
  ADD COLUMN IF NOT EXISTS seo_meta_desc          text,
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
```

```
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS schema_json_ld         jsonb;

-- 6. industries: canonical + cycling + faq_list + schema
--    (seo_title and seo_meta_desc already present)
ALTER TABLE public.industries
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS schema_json_ld         jsonb;

-- 7. free_tools: SEO fields + cycling + faq_list + schema
ALTER TABLE public.free_tools
  ADD COLUMN IF NOT EXISTS seo_title              text,
  ADD COLUMN IF NOT EXISTS seo_meta_desc          text,
  ADD COLUMN IF NOT EXISTS canonical_url          text,
  ADD COLUMN IF NOT EXISTS keyword_cycling_blocks jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS faq_list               jsonb DEFAULT '[]'::jsonb,
  ADD COLUMN IF NOT EXISTS schema_json_ld         jsonb;

-- 8. knowledge_graph_entities: deduplication keys + indexes
ALTER TABLE public.knowledge_graph_entities
  ADD COLUMN IF NOT EXISTS wikidata_qid   text,
  ADD COLUMN IF NOT EXISTS wikipedia_slug text;

CREATE UNIQUE INDEX IF NOT EXISTS idx_kg_wikidata_qid
  ON public.knowledge_graph_entities (wikidata_qid)
  WHERE wikidata_qid IS NOT NULL;

CREATE UNIQUE INDEX IF NOT EXISTS idx_kg_wikipedia_slug
  ON public.knowledge_graph_entities (wikipedia_slug)
  WHERE wikipedia_slug IS NOT NULL;

-- 9. GIN indexes: about/mentions on pages, blog_posts, pseo
CREATE INDEX IF NOT EXISTS idx_pages_about_entities
  ON public.pages USING GIN (about_entities);
CREATE INDEX IF NOT EXISTS idx_pages_mentions_entities
  ON public.pages USING GIN (mentions_entities);
CREATE INDEX IF NOT EXISTS idx_blog_posts_about_entities
  ON public.blog_posts USING GIN (about_entities);
CREATE INDEX IF NOT EXISTS idx_blog_posts_mentions_entities
  ON public.blog_posts USING GIN (mentions_entities);
CREATE INDEX IF NOT EXISTS idx_pseo_about_entities
  ON public.pseo_page_instances USING GIN (about_entities);
CREATE INDEX IF NOT EXISTS idx_pseo_mentions_entities
  ON public.pseo_page_instances USING GIN (mentions_entities);
```

```
-- 10. Create public.people table
CREATE TABLE IF NOT EXISTS public.people (
  id                      uuid NOT NULL DEFAULT uuid_generate_v4(),
  business_id             uuid NOT NULL,
  full_name               text NOT NULL,
  slug                    text NOT NULL,
  job_title               text,
  bio                     text,
  short_bio               text,
  profile_image_url       text,
  profile_image_alt       text,
  email                   text,
  website_url             text,
  linkedin_url            text,
  sameas_urls             jsonb DEFAULT '[]'::jsonb,
  social_links            jsonb,
  expertise_areas         text[] DEFAULT '{}'::text[],
  credentials             text[] DEFAULT '{}'::text[],
  works_for_business_id   uuid,
  is_author               boolean DEFAULT true,
  is_reviewer             boolean DEFAULT false,
  is_team_member          boolean DEFAULT true,
  published               boolean DEFAULT false,
  seo_title               text,
  seo_meta_desc           text,
  canonical_url           text,
  in_language             text DEFAULT 'en-GB',
  schema_json_ld          jsonb,
  created_at              timestamptz DEFAULT now(),
  last_updated            timestamptz DEFAULT now(),
  CONSTRAINT people_pkey PRIMARY KEY (id),
  CONSTRAINT people_business_id_fkey
    FOREIGN KEY (business_id) REFERENCES public.businesses(id),
  CONSTRAINT people_works_for_fkey
    FOREIGN KEY (works_for_business_id) REFERENCES public.businesses(id),
  CONSTRAINT people_slug_business_unique UNIQUE (business_id, slug)
);

-- 11. blog_posts: add author/reviewer FK constraints (after people table created)
ALTER TABLE public.blog_posts
  ADD CONSTRAINT IF NOT EXISTS blog_posts_author_person_fkey
    FOREIGN KEY (author_person_id) REFERENCES public.people(id),
  ADD CONSTRAINT IF NOT EXISTS blog_posts_reviewer_person_fkey
    FOREIGN KEY (reviewer_person_id) REFERENCES public.people(id);

-- ============================================================
```

```
-- BACKFILL (run after above, validate before dropping legacy cols)
-- ==============================================================

-- Backfill faq_list from pages.faqs (if jsonb array)
UPDATE public.pages
  SET faq_list = faqs
  WHERE faqs IS NOT NULL AND faq_list = '[]'::jsonb;

-- Backfill faq_list from blog_posts."FAQs"
UPDATE public.blog_posts
  SET faq_list = "FAQs"
  WHERE "FAQs" IS NOT NULL AND faq_list = '[]'::jsonb;

-- Backfill faq_list from pseo_page_instances.unique_faqs
UPDATE public.pseo_page_instances
  SET faq_list = unique_faqs
  WHERE unique_faqs IS NOT NULL AND unique_faqs != '[]'::jsonb
    AND faq_list = '[]'::jsonb;

-- ==============================================================
-- DROP LEGACY COLUMNS (only after Phase 3 generators complete
-- and schema_json_ld + faq_list validated in all code paths)
-- ==============================================================

-- ALTER TABLE public.pages DROP COLUMN IF EXISTS faqs;
-- ALTER TABLE public.pages DROP COLUMN IF EXISTS generated_schema_markup;
-- ALTER TABLE public.blog_posts DROP COLUMN IF EXISTS "FAQs";
-- ALTER TABLE public.blog_posts DROP COLUMN IF EXISTS generated_schema_markup;
-- ALTER TABLE public.pseo_page_instances DROP COLUMN IF EXISTS unique_faqs;
-- ALTER TABLE public.pseo_page_instances DROP COLUMN IF EXISTS generated_schema_markup;

-- End of migration
```

6. Schema Architecture Amendments (amends Section B)

The following updates apply to Section B of the content model. All occurrences of generated_schema_markup should be read as schema_json_ld. All occurrences of faqs / FAQs / unique_faqs in the context of FAQPage generation should be read as faq_list.

6.1 Layer 5 — FAQPage schema: updated source column

Updated rule:
FAQPage schema is generated from faq_list (jsonb) on whichever content table is being processed. The generator reads the same column name regardless of whether the source is pages, blog_posts, services, pseo_page_instances, industries, or free_tools. No branching logic per table is required.

6.2 Layer 3 — BlogPosting author resolution

Updated rule:
BlogPosting Person schema resolves as follows: (1) If blog_posts.author_person_id is non-null, fetch the people record and use full_name, job_title, website_url, profile_image_url, sameas_urls for the Person object. (2) If author_person_id is null, fall back to author_name / author_url / author_image_url fields. This allows legacy posts to function while new posts use the people table.

6.3 Schema cache — column name

Updated rule:
The generated JSON-LD cache is stored in schema_json_ld (jsonb) on all content tables. The front end reads this column verbatim and emits it as a <script type='application/ld+json'> block in the page <head>. No schema assembly occurs at render time.

6.4 Layer 2 — Organization vs associates vs people

Updated rule:
associates = external brands, partner organisations, and software platforms. Use in brand, sponsor, provider roles in schema where the entity is an external organisation. Do NOT use for human author attribution. people = human authors, reviewers, and team members. Use for Person schema in BlogPosting.author, BlogPosting.reviewedBy, AboutPage team section.

7. Component Variant Note (answer to pre-build question)

Question: do we need to finalise hero/component variants before designing the content model?

Answer: No. The content model defines data fields, not visual presentation. The same field schema (headline, subheadline, body copy, CTA pair, image, trust signals, stat items etc.) serves all visual variants of a component. A display_variant selector field is the only addition needed at content model level.

7.1 What to add to the content model for component variants

Field Name	Field Type	Required?	Notes
display_variant	Select string	No	Which visual layout to use. Values defined per component family (e.g. 'conversion' | 'proof-led' | 'image-split' for heroes). If null, component uses its default variant.
stat_items[]	JSON array	No	Variant-specific optional field for proof-led hero and stats/metrics components. Array of { label, value } objects. Marked optional — variants that don't use it leave it empty.
image_position	Select	No	Variant-specific for image-split layouts. Values: 'left' | 'right'. Default: 'right'.
background_variant	Select	No	Dark / light / brand background. Optional on all components.

7.2 When to lock variant choices

Timing:
Lock the actual variant choice per template (which hero variant goes on the Service Category page, etc.) BEFORE Phase 4 Stitch build — this is a design decision, not a DB decision. It can be made in parallel with Phase 2. Task 4A.0 in the roadmap is the formal lock point. Component fields are variant-agnostic so Phase 2 DB work can proceed now without waiting.

7.3 Minimum variant targets (3/2/1 rule)

Per the component variant plan:

Component	Variants	Notes
Hero sections	**3 variants**	Conversion / Proof-led / Image-split — top-of-page, high impact
CTA sections	**3 variants**	Simple banner / Split with image / Proof + CTA — used on all page types
Feature/service highlights	**3 variants**	Icon card grid / Text rows with checkmarks / Alternating image+feature rows
Testimonials/proof	**3 variants**	Single featured quote / 3-card grid / Review strip
FAQ accordion	**2 variants**	Standard accordion / Two-column Q+A layout
Process/steps/timeline	**2 variants**	Vertical step list / Horizontal timeline

Component	Variants	Notes
Author/expert block	**2 variants**	Compact author card / Expanded credibility panel (reads people record)
Local proof block (PSEO)	**2 variants**	Landmarks + service relevance / Why-choose-us proof panel
Related resources cards	**2 variants**	Card grid / List with descriptions
Case study highlight	**2 variants**	Problem → Action → Result cards / Outcome summary with KPI tiles + quote
Stats/outcomes	**2 variants**	Simple stat cards / Big-number strip + text
Comparison blocks	**2 variants**	Comparison table / Side-by-side cards
Tool explainer	**2 variants**	How-to-use step list / Input-output explainer with examples
Pricing cards	**2 variants**	Tiered pricing cards / Comparison pricing table — if in scope
Breadcrumbs	**1 variant**	Utility — no visual variation needed at MVP
Schema head injector	**1 variant**	Structural/invisible — reads schema_json_ld, emits JSON-LD script
Breadcrumbs, embed wrapper, TOC, contact panel, dividers	**1 each**	Utility components — variation adds no value at MVP stage